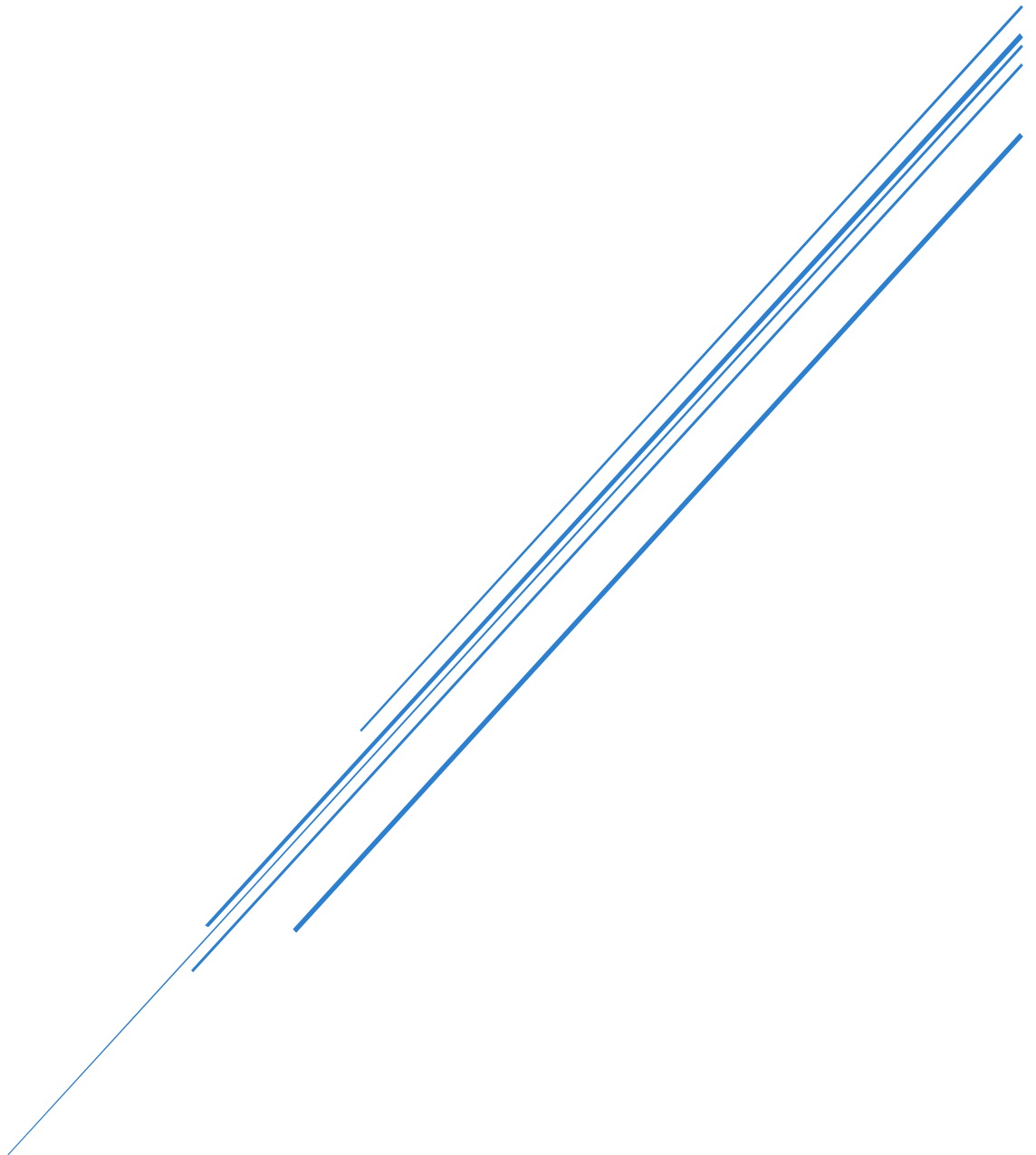


LAB 7 ANSWERS REPORT

Oliver Wuttke – WUTT0019

Date of Completion: 29/09/2026



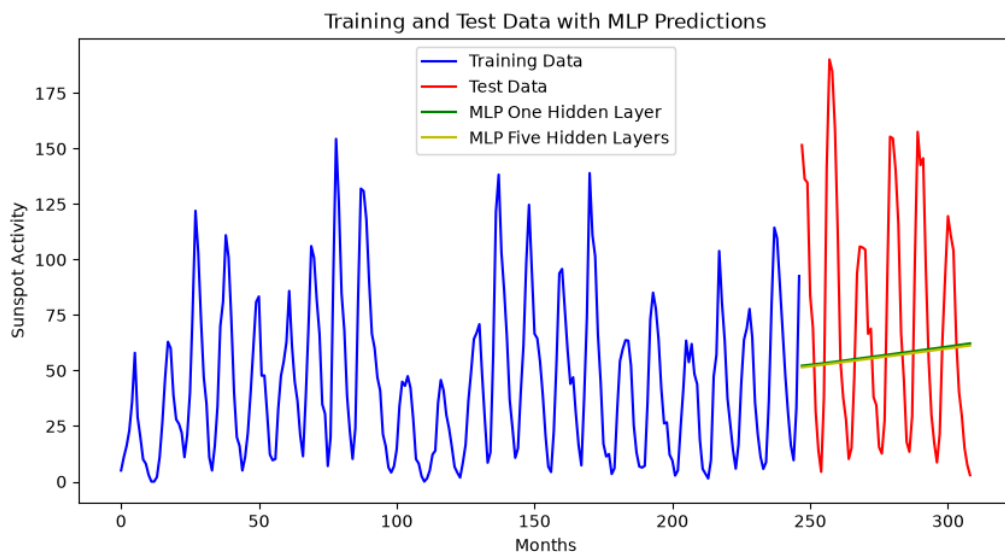
Flinders University – Tonsley
Artificial Intelligence (COMP3742)

Table of Contents

Question 1	2
Using scikit-learn fit a MLP to the sunspot data using ReLU activation function and the adam solver. Evaluate the change in Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) as the number of hidden layers is increased from 1 to 5 with 50 nodes per hidden layer. Plot and discuss the results.....	2
Question 2	3
Using scikit-learn fit a 2 hidden layer MLP to the sunspot data using ReLU activation function and the adam solver. Evaluate the change in RMSE and MAE as the number of nodes in each layer is increased from 10 to 300. Plot and discuss the results.....	3
Question 3	4
Using scikit-learn fit a 3 hidden layer MLP (50 hidden nodes per layer) to the sunspot data using ReLU activation function and the adam solver. Evaluate the change in RMSE and MAE using at least 3 different activation functions. Plot and discuss the results.	4
Question 4	5
Using scikit-learn fit a 2 hidden layer MLP (50 hidden nodes per layer) to the sunspot data using ReLU activation function and the adam solver. Perform Grid Search hyperparameter optimization for different solvers, batch sizes, and learning rates using cross-validation. Compare RMSE, and MAE between the unoptimized and optimized models.....	5
Question 5	5
Implement an MLP architecture (two hidden layers with 50 and 25 nodes respectively, both layers containing 'tanh' activation) using PyTorch. Train it on the same sunspot dataset and evaluate its performance.....	5

Question 1

Using scikit-learn fit a MLP to the sunspot data using ReLU activation function and the adam solver. Evaluate the change in Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) as the number of hidden layers is increased from 1 to 5 with 50 nodes per hidden layer. Plot and discuss the results.



=== MLP with 1 Hidden Layer ===

MAE: 46.50481903226155

RMSE: 56.07535640353801

=== MLP with 5 Hidden Layers ===

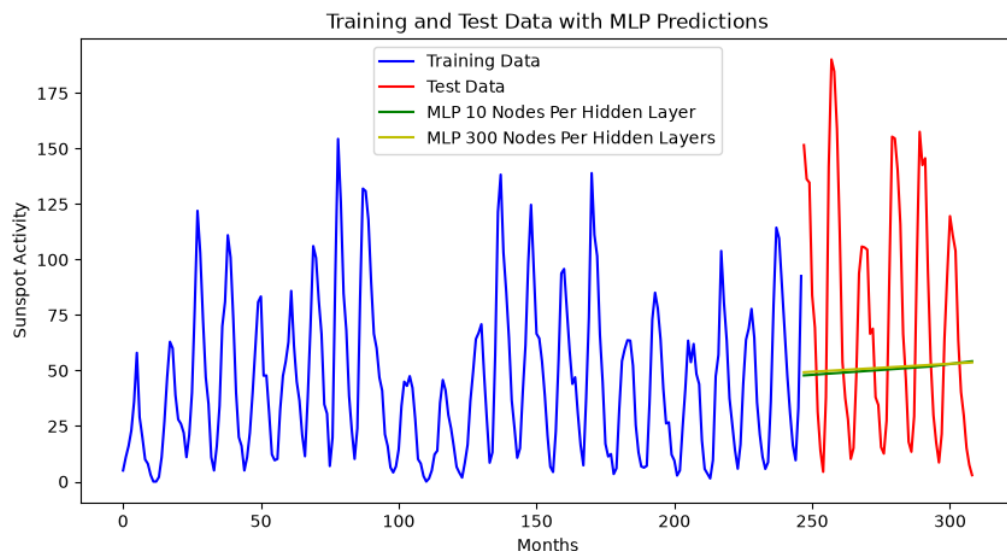
MAE: 46.548283023548976

RMSE: 56.27225319565861

The above results show that there's very little difference between the 5 hidden layer MLP model and the one hidden layer. Their errors are near identical with the single layer model performing slightly better. They both just predict straight lines.

Question 2

Using scikit-learn fit a 2 hidden layer MLP to the sunspot data using ReLU activation function and the adam solver. Evaluate the change in RMSE and MAE as the number of nodes in each layer is increased from 10 to 300. Plot and discuss the results.



=== MLP with 10 Nodes for Each Hidden Layer ===

MAE: 47.03070816111727

RMSE: 58.037614896184856

=== MLP with 300 Nodes for Each Hidden Layer ===

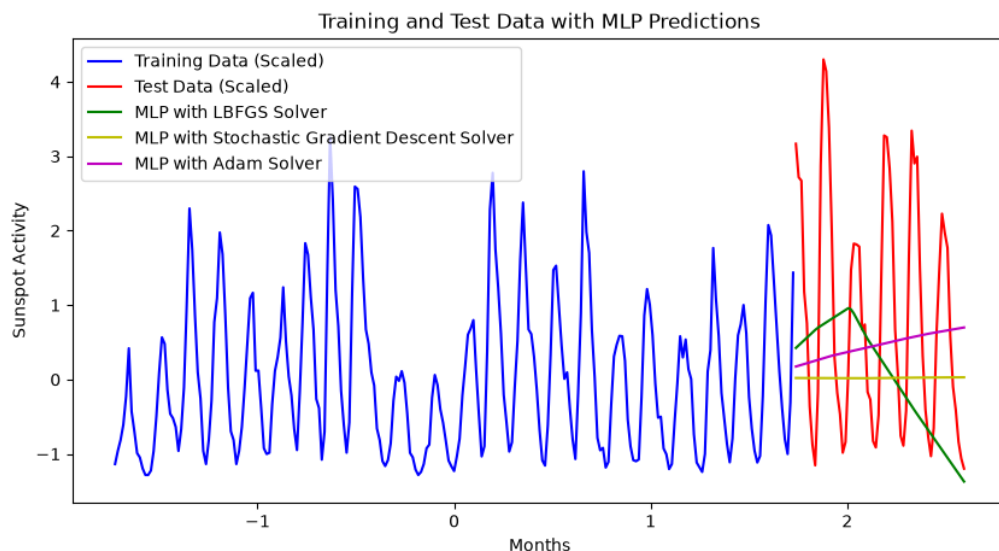
MAE: 46.84401017660735

RMSE: 57.59125827185336

We see very similar results to question 1 here with near identical errors and plots. Nothing new to comment. Maybe it's got something to do with the non-linear nature of the data and that the neural network is piecewise linear with a few relu functions.

Question 3

Using scikit-learn fit a 3 hidden layer MLP (50 hidden nodes per layer) to the sunspot data using ReLU activation function and the adam solver. Evaluate the change in RMSE and MAE using at least 3 different activation functions. Plot and discuss the results.



=== Results for lbfgs solver ===

MAE: 1.447201605104046

RMSE: 1.7935684658475182

=== Results for sgd solver ===

MAE: 1.401190977220296

RMSE: 1.7731412847829935

=== Results for adam solver ===

MAE: 1.3729171518209309

RMSE: 1.6459175613217307

The results are more interesting here since we also scaled both the target and features. But we barely see any difference in the actual model performance. The lower MAE and RMSE are deceiving because of the scaled nature of our data.

Question 4

Using scikit-learn fit a 2 hidden layer MLP (50 hidden nodes per layer) to the sunspot data using ReLU activation function and the adam solver. Perform Grid Search hyperparameter optimization for different solvers, batch sizes, and learning rates using cross-validation. Compare RMSE, and MAE between the unoptimized and optimized models.

=== Unoptimized MLP Model ===

MAE: 47.0549032624418

RMSE: 55.49939092105228

=== Optimized MLP Model ===

MAE: 48.17271086620081

RMSE: 61.66504162546795

Optimal Params:

{'batch_size': 64, 'learning_rate': 'invscaling', 'learning_rate_init': 0.01, 'solver': 'sgd'}

These results are basically identical again to the previous questions. It's pointing to the fact that without lagging our features or doing something else than just fitting an MLP model to the features and label that we won't see better results.

Question 5

Implement an MLP architecture (two hidden layers with 50 and 25 nodes respectively, both layers containing 'tanh' activation) using PyTorch. Train it on the same sunspot dataset and evaluate its performance.

Test MSE: 3562.11

Test RMSE: 59.68

Test MAE: 47.44

And finally nothing new with hand rolling the neural net in PyTorch besides having to revisit some old notebooks to jog my memory on how to use PyTorch.